ENTERPRISE TECHNICAL GUIDEBOOK

Multi-Cloud Observability Platform

From Evaluation to Production

AWS Landing Zone | OpenTelemetry | New Relic
EKS · GKE · AKS | Amazon Bedrock AI | Kubernetes

Evaluate	Design	Implement	Operate
Discovery & gap analysis	Architecture & cost model	Step-by-step deployment	operations & AI

Saroj Naik | Principal Cloud Architect | linkedin.com/in/nsaroj

Table of Contents

1. **Executive Summary**
   What this platform delivers and for whom
2. **Phase 1 — Current State Evaluation**
   Assessment framework, discovery questions, gap analysis
3. **Phase 2 — High Level Design**
   Architecture principles, component selection, cost modelling
4. **Phase 3 — Low Level Design**
   Data flows, RBAC model, retention policies
5. **Phase 4 — Action Plan**
   Sprint plan, team roles, risk register
6. **Phase 5 — Production Implementation**
   AWS Landing Zone, OTel, EKS/GKE/AKS step by step
7. **New Relic Configuration**
   NRQL queries, dashboards, alert policies, SLOs
8. **AI-Powered Operations**
   Bedrock, incident triage, anomaly detection
9. **Security and Compliance**
   RBAC, secrets, encryption, audit trails
10. **Operations Runbook**
    Day-2 operations, scaling, troubleshooting
11. **Author & Contact**

1 Executive Summary

What this platform delivers and for whom

Purpose of This Document

This guidebook describes how to design, plan, and implement a production-grade multi-cloud observability platform across AWS EKS, Google GKE, and Azure AKS. It is structured as a five-phase engagement — from initial evaluation through live production operation. Each phase has clear deliverables, decision points, and implementation steps that any platform engineering team can follow.

Who This Is For

Role	How to Use This Document
CTO / VP Engineering	Understand the platform value, cost, and timeline
Platform Architect	All chapters: design decisions, configurations, and trade-offs
SRE / Platform Engineer	Implementation steps and operational runbooks
DevOps Engineer	Deployment guides and pipeline integration
Security Engineer	RBAC, encryption, and compliance controls

What the Platform Delivers

Modern distributed systems running across multiple clouds generate enormous volumes of telemetry. Without a unified observability platform, teams spend hours correlating data across siloed tools, miss performance degradation before customers notice, and cannot trace a request across microservices. This platform eliminates those gaps.

Outcome	Measurement	Without This Platform
Unified visibility	Single dashboard across AWS, GCP, Azure	3 separate tools, no correlation
Faster incident resolution	MTTR reduced from 45 min to 8 min	Manual log grep across clusters
Proactive detection	Alerts before user impact	Reactive — users report issues first
AI-assisted triage	Root cause in plain English	Engineers interpret raw metrics
Cost visibility	Per-team, per-cluster attribution	No cloud cost breakdown by team
Compliance evidence	Audit trail for all telemetry access	Manual evidence collection

Engagement Timeline — Five Phases

1 **Phase 1: Evaluation** | Week 1–2
Current state assessment | Gap analysis | Tool inventory | Stakeholder interviews

2 **Phase 2: High Level Design** | Week 2–3
Architecture decision | Component selection | Cost modelling | Executive sign-off

3 **Phase 3: Low Level Design** | Week 3–5
Detailed configs | RBAC model | Retention policies | Runbook templates

4 **Phase 4: Action Plan** | Week 5–6
Sprint planning | Team assignments | Risk register | Dependency mapping

5 **Phase 5: Production Implementation** | Week 6–14
Infrastructure | Instrumentation | Dashboards | Alerts | Handover

1 Phase 1 — Current State Evaluation

Assessment framework, discovery questions, gap analysis

Why Evaluation Comes First

Skipping evaluation is the most common reason observability projects fail. Teams jump to tool selection before understanding what they actually need to observe, who needs to see it, and what existing data exists. A two-week evaluation prevents six months of rework.

Discovery Questions — Ask These First

Area	Questions to Answer
Current tooling	What monitoring tools exist? Are they paid or OSS? What are the contract terms?
Data volumes	How many services? Estimated spans/min, metrics/min, log lines/min?
Cloud footprint	Which clouds and regions? Any on-premises workloads?
Team structure	Who owns observability? How many SREs? On-call rotation size?
Pain points	Top 3 incidents in last 90 days. How long did each take to resolve?
Compliance	SOC 2? HIPAA? PCI? Data residency requirements?
Budget	Annual observability budget? Who approves new tooling spend?
SLOs	Are SLOs defined? Are they measured? Are they published to users?

Gap Analysis Maturity Matrix

Rate each capability from Level 1 to Level 4. Anything below Level 3 is a gap to address in the implementation plan.

Capability	Level 1 — None	Level 2 — Basic	Level 3 — Production	Level 4 — Optimised
Metrics	No metrics collected	Per-service dashboards	Cross-service alerting	SLO-based burn rate alerts
Logs	stdout only	Centralised log aggregation	Structured JSON + search	Correlated to traces
Traces	No tracing	Manual instrumentation	Auto-instrumentation	Distributed trace UI
Synthetics	Manual testing	Ping monitors only	Scripted API tests	Full user journey tests
AI / Anomaly	No detection	Static thresholds	ML-based anomaly	Plain-English AI triage

Capability	Level 1 — None	Level 2 — Basic	Level 3 — Production	Level 4 — Optimised
Cost visibility	No attribution	Per-cloud billing	Per-service costs	FinOps dashboards

Evaluation Deliverables

- Current state report — one page per tool in use today
- Gap analysis matrix — completed version of the table above
- Data volume estimates — spans, metrics, logs per minute per cluster
- Stakeholder map — who owns observability decisions and budget
- Risk register — top 5 risks to the implementation with mitigations
- Go/No-Go recommendation — with rationale and cost estimate

2 Phase 2 — High Level Design

Architecture principles, component selection, cost modelling

Architecture Principles

> *Vendor-neutral instrumentation: All applications use OpenTelemetry SDK. Switching backends requires only a gateway config change — no re-instrumentation of applications.*
>
> *Single source of truth: New Relic NRDB is the unified backend. No per-team Grafana instances, no siloed Prometheus. One query language for all signals.*
>
> *Data stays in your account: Amazon Bedrock AI runs inside your AWS account. No telemetry, logs, or security findings leave your VPC boundary.*
>
> *GitOps everything: All collector configs, dashboards, and alert policies are version-controlled. No manual configuration that cannot be audited.*
>
> *Least privilege always: IRSA scopes AWS permissions to individual service accounts. Each team sees only their own namespace data in dashboards.*

Component Selection — With Rationale

Component	Selected Tool	Alternatives Evaluated	Decision Rationale
Instrumentation	OpenTelemetry SDK	Datadog Agent, NR Agent	Vendor-neutral. Backend portable.
Node collection	OTel DaemonSet	Fluentd, Filebeat	Single agent for all 3 signals.
Gateway	OTel Collector Gateway	Kafka, Logstash, Vector	Native OTLP. Tail sampling built-in.
Metrics/Trace backend	New Relic NRDB	Datadog, Grafana Cloud	Managed SaaS. Sub-second NRQL.
Log backend	New Relic Logs	Splunk, Elastic, Loki	Unified with metrics and traces.
Synthetics	New Relic Synthetics	Pingdom, Checkly	Integrated with NR alerting + SLOs.
AI analysis	Amazon Bedrock	OpenAI API, Anthropic direct	Data stays in VPC. IAM auth.
K8s metrics	kube-state-metrics	cAdvisor only	Deployment/quota/PVC state exposed.

Cost Model — Monthly Estimates

Component	Small (10 services)	Medium (50 services)	Large (200+ services)
New Relic ingest	~$0 (under 100GB free)	~$200–500/month	~$800–2000/month
AWS Bedrock AI calls	~$5–10/month	~$20–50/month	~$80–150/month
OTel Collector nodes	~$30/month	~$60/month	~$150/month
New Relic Synthetics	Included in plan	Included in plan	~$50–200/month
kube-state-metrics	$0 (on existing nodes)	$0	$0
TOTAL ESTIMATE	~$35–50/month	~$300–650/month	~$1100–2400/month

3 Phase 3 — Low Level Design

Data flows, RBAC model, sampling strategy, retention policies

End-to-End Data Flow

Step	Component	Action	Output
1	OTel SDK in app	Auto-instrument HTTP, DB, Redis, gRPC	Spans, metrics, logs via OTLP port 4317
2	OTel Agent DaemonSet	Receive OTLP, add K8s metadata, pre-batch	Enriched telemetry to Gateway
3	OTel Gateway	Tail sample, enrich, route by landing-zone tag	Batched OTLP to New Relic
4	New Relic NRDB	Store all signals, index for NRQL queries	Queryable within 5 seconds
5	Bedrock Lambda	Read anomalies from NR API, call Claude	Plain-English summary to Slack
6	Synthetics	Check endpoints every 1 min from 20+ locations	Availability SLI independent of app

Sampling Strategy — Tail-Based

Tail-based sampling at the Gateway decides what to keep after the full trace is complete. This is critical — head-based sampling at the SDK level would discard error traces before knowing they are errors.

Rule	Condition	Action	Rationale
Keep errors	status_code = ERROR	Keep 100%	Never miss a bug in production
Keep slow traces	duration greater than 500ms	Keep 100%	Catch latency issues proactively
Keep outliers	duration greater than 2x p99 baseline	Keep 100%	Statistical anomalies captured
Sample normal	All other healthy traces	Keep 10%	Cost control with good coverage

RBAC Model — Who Sees What

Role	New Relic Access	Namespace Scope	Alert Management
Platform Admin	Full account access	All namespaces	Create, edit, delete
Team Lead	Read + dashboard create	Own namespace only	View + acknowledge
Developer	Read only	Own namespace only	View only

Role	New Relic Access	Namespace Scope	Alert Management
SRE On-Call	Full read + silence alerts	All namespaces	Silence + acknowledge
Security	Read all logs + traces	All namespaces	View only
FinOps	Cost dashboards only	N/A	None

Data Retention Policies

Signal Type	Hot Retention	Cold Retention	Cost Driver
Metrics (raw)	13 months in New Relic	N/A — NR manages	Data ingest volume
Traces	8 days (standard)	90 days (Data Plus)	Span count x duration
Logs	30 days (standard)	1 year (Data Plus)	Log line volume
Synthetic results	13 months	N/A	Monitor count + frequency
Events / Alerts	13 months	N/A	Event count

4 Phase 4 — Action Plan

Sprint plan, team roles, dependencies, risk register

Sprint Plan — 8 Weeks to Production

Sprint	Weeks	Work Items	Owner	Dependency
0 — Setup	1	New Relic account, AWS Landing Zone, Terraform state bucket	Platform Architect	AWS account access
1 — Foundation	2–3	OTel Collector Gateway, IAM roles, K8s namespaces	Platform Engineer	Sprint 0 complete
2 — EKS	3–4	OTel DaemonSet on EKS, kube-state-metrics, first NR data flowing	SRE	Sprint 1 complete
3 — GKE + AKS	4–5	Repeat DaemonSet for GKE and AKS, cross-cloud tagging	SRE	Sprint 2 complete
4 — Dashboards	5–6	Platform overview, per-team, infrastructure dashboards	Platform Engineer	Sprint 3 complete
5 — Alerts	6–7	Alert policies, runbooks, PagerDuty and Slack integration	SRE + DevOps	Sprint 4 complete
6 — Synthetics	7	Ping, API, and scripted browser monitors, SLO configuration	QA + SRE	Sprint 5 complete
7 — AI + Handover	8	Bedrock Lambda, documentation, team training, go-live	All	Sprint 6 complete

Team Roles and Responsibilities

Role	Count	Primary Responsibility	Skills Required
Platform Architect	1	Architecture decisions, cost, stakeholder alignment	Kubernetes, AWS, Terraform, New Relic
Platform Engineer	2	OTel deployment, Terraform IaC, GitOps management	Kubernetes, Helm, Python, Bash
SRE	1	Alert design, runbooks, on-call integration	Observability, incident management
App Developer (per team)	1 per team	SDK instrumentation, custom spans for business logic	Python/Java/Node, OTel SDK
Security Engineer	1 part-time	RBAC review, secrets management, compliance evidence	IAM, Vault, audit logging

Risk Register

Risk	Probability	Impact	Mitigation
App teams resist instrumentation	Medium	High	Auto-instrumentation requires zero code change for 80% coverage
Data volume exceeds NR free tier	High	Medium	Implement tail sampling at gateway before New Relic ingest
GKE/AKS network blocks gateway	Medium	High	Use TLS + allowlist gateway IP in network policies
Bedrock model access delayed	Low	Low	AI summariser is optional — platform works without it
Key engineer leaves mid-project	Low	High	All configs in Git. Runbooks written as part of each sprint.
New Relic cost overrun	Medium	Medium	Set NR data drop rules at 90GB/month to stay in free tier

5 Phase 5 — Production Implementation

AWS Landing Zone, OTel, EKS/GKE/AKS step by step

Step 1 — Create Observability Namespace and Secrets

```
# Create observability namespace on your EKS cluster
kubectl create namespace observability

# Store New Relic license key as Kubernetes secret
kubectl create secret generic newrelic-license \
  --from-literal=license_key=YOUR_NR_LICENSE_KEY \
  --namespace observability

# Verify secret was created
kubectl get secret newrelic-license -n observability
```

Step 2 — Deploy OTel Collector Gateway

The Gateway is the central aggregation point. Deploy as a 3-replica Deployment for high availability. Anti-affinity rules ensure no two replicas land on the same node.

```
# File: gitops/infrastructure/observability/otel-gateway-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-gateway
  namespace: observability
spec:
  replicas: 3
  selector:
    matchLabels:
      app: otel-gateway
  template:
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values: [otel-gateway]
              topologyKey: kubernetes.io/hostname
      containers:
        - name: otel-gateway
          image: otel/opentelemetry-collector-contrib:0.96.0
          args: ["--config=/conf/gateway-config.yaml"]
          env:
            - name: NEW_RELIC_LICENSE_KEY
              valueFrom:
                secretKeyRef:
                  name: newrelic-license
                  key: license_key
          resources:
            requests: { memory: "1Gi", cpu: "500m" }
```

```
            limits:   { memory: "2Gi", cpu: "1000m" }
```

Gateway ConfigMap — Tail Sampling + New Relic Export

```
processors:
  tail_sampling:
    decision_wait: 10s
    num_traces: 50000
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: keep-slow
        type: latency
        latency: {threshold_ms: 500}
      - name: sample-10-percent
        type: probabilistic
        probabilistic: {sampling_percentage: 10}

exporters:
  otlphttp/newrelic:
    endpoint: "https://otlp.nr-data.net"
    headers:
      api-key: "${NEW_RELIC_LICENSE_KEY}"
    compression: gzip
    retry_on_failure:
      enabled: true
      max_elapsed_time: 300s
```

Step 3 — Deploy OTel Agent DaemonSet on Each Cluster

Run this for EKS first, then switch kubectl context to GKE and AKS and repeat. The only difference between clusters is the cloud_provider attribute value in the ConfigMap.

```
# Deploy on EKS
kubectl config use-context YOUR_EKS_CONTEXT
kubectl apply -f gitops/infrastructure/observability/otel-agent-daemonset.yaml
kubectl get pods -n observability -l app=otel-agent
# Expected: one pod per node, all Running

# Switch to GKE — update cloud_provider to 'gcp' in ConfigMap first
kubectl config use-context YOUR_GKE_CONTEXT
kubectl apply -f gitops/infrastructure/observability/otel-agent-daemonset.yaml

# Switch to AKS — update cloud_provider to 'azure' in ConfigMap first
kubectl config use-context YOUR_AKS_CONTEXT
kubectl apply -f gitops/infrastructure/observability/otel-agent-daemonset.yaml

# Verify data flowing in New Relic — run this NRQL in NR Query Builder:
# SELECT count(*) FROM Metric WHERE cloud_provider IN ('aws','gcp','azure')
# Should return > 0 within 5 minutes of deploying all agents
```

Step 4 — Instrument Applications (Python / FastAPI)

```
# requirements.txt — add these packages
opentelemetry-api==1.22.0
```

```
opentelemetry-sdk==1.22.0
opentelemetry-instrumentation-fastapi==0.43b0
opentelemetry-exporter-otlp-proto-grpc==1.22.0

# otel_setup.py — call init_otel(app) in your main.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.sdk.resources import Resource
import os

def init_otel(app):
    resource = Resource.create({
        'service.name':           os.getenv('SERVICE_NAME'),
        'service.version':        os.getenv('APP_VERSION', '1.0.0'),
        'deployment.environment': os.getenv('ENVIRONMENT', 'prod'),
        'cloud.provider':         os.getenv('CLOUD_PROVIDER', 'aws'),
        'team':                   os.getenv('TEAM_NAME'),
    })
    provider = TracerProvider(resource=resource)
    provider.add_span_processor(BatchSpanProcessor(
        OTLPSpanExporter(
            endpoint=os.getenv('OTEL_ENDPOINT', 'http://localhost:4317'),
            insecure=True,
        ))
    )
    trace.set_tracer_provider(provider)
    FastAPIInstrumentor.instrument_app(app)  # auto-instruments all routes
    return trace.get_tracer(__name__)
```

New Relic Configuration

NRQL queries, dashboards, alert policies, SLOs

Account Setup

- Sign up free at newrelic.com — 100 GB/month ingest included at no cost
- Go to: Account Settings → API keys → Create key → Type: License → Copy the key
- Store as K8s secret: kubectl create secret generic newrelic-license --from-literal=license_key=YOUR_KEY -n observability
- Note your Account ID from the URL: one.newrelic.com/accounts/[ACCOUNT_ID]

Essential NRQL Queries — Copy and Run in New Relic

Error rate across all clouds

```
SELECT percentage(count(*), WHERE error IS true) as 'Error %'
FROM Transaction
FACET cloudProvider, appName
SINCE 1 hour ago TIMESERIES 5 minutes
```

P99 latency by service and cluster

```
SELECT percentile(duration, 99) as 'P99 ms'
FROM Transaction
FACET service.name, cluster_name
SINCE 1 hour ago
```

Pod health across all clusters

```
SELECT uniqueCount(k8s.pod.name)                                    as 'Total',
  filter(uniqueCount(k8s.pod.name), WHERE status = 'Running')       as 'Running',
  filter(uniqueCount(k8s.pod.name), WHERE status = 'Failed')        as 'Failed'
FROM K8sPodSample
FACET clusterName SINCE 5 minutes ago
```

Cross-cloud latency comparison

```
SELECT percentile(duration, 50, 95, 99)
FROM Span
WHERE cloud.provider IN ('aws','gcp','azure')
FACET cloud.provider
SINCE 1 hour ago TIMESERIES 5 minutes
```

Alert Policy — Ready-to-Use Conditions

Alert Name	NRQL Condition	Critical	Warning
High Error Rate	percentage(count(*), WHERE error IS true) FROM Transaction	> 5 %	> 2 %
High P99 Latency	percentile(duration,99)*1000 FROM Transaction	> 2000 ms	> 1000 ms

Alert Name	NRQL Condition	Critical	Warning
Pod CrashLoop	count(*) FROM K8sPodSample WHERE status='CrashLoopBackOff'	>= 1	N/A
High Node CPU	average(cpuPercent) FROM SystemSample	> 90 %	> 75 %
High Memory	average(memoryUsedPercent) FROM SystemSample	> 90 %	> 80 %
OTel Queue Full	average(otelcol_exporter_queue_size) FROM Metric	> 4500	> 3000
Synthetic Failure	percentage(count(*), WHERE result='FAILED') FROM SyntheticCheck	> 0 %	N/A

SLO Configuration

Define three SLOs per critical service — availability, latency, and error rate. These queries power the SLO dashboard in New Relic.

Availability SLO — 99.9% over 28 days

```
SELECT percentage(count(*), WHERE error IS false)
FROM Transaction
WHERE appName = 'YOUR_SERVICE_NAME'
SINCE 28 days ago
-- Target: 99.9%   |   Error budget: 40.3 minutes allowed downtime
```

Latency SLO — 99% of requests under 500ms

```
SELECT percentage(count(*), WHERE duration < 0.5)
FROM Transaction
WHERE appName = 'YOUR_SERVICE_NAME'
SINCE 28 days ago
-- Target: 99%    |   Error budget: 7.2 hours per 28-day window
```

AI-Powered Operations

Amazon Bedrock, incident triage, anomaly detection

Why AI in Observability

Traditional alerting fires when a metric crosses a static threshold. AI fires when something unusual happens — even if it has never happened before. More importantly, AI translates raw telemetry into plain English that an on-call engineer can act on at 3am without needing to be an expert in every service.

Use Cases — Prioritised by Impact

Use Case	Input	Output	Value	Effort
Security finding summariser	Bandit + Trivy JSON reports	Plain English fixes per finding	High	Low — 1 day
Incident triage	CloudWatch alert + recent logs	Root cause + 3 diagnostic steps	High	Medium — 3 days
Anomaly explanation	Metric spike data	This looks like X because Y	Medium	Medium — 3 days
Cost anomaly detection	Daily AWS cost delta	Specific optimisation recommendations	Medium	Medium — 2 days
Runbook generation	Historical incident data	Draft runbook per alert type	High	High — 1 week

Bedrock Setup — One Time

- AWS Console → Amazon Bedrock → Model access → Manage model access
- Tick: Anthropic → Claude 3.5 Sonnet v2 → Submit use case form (approval is usually instant)
- Attach the IAM policy below to your Lambda or CI/CD role

```
# Minimum IAM policy for Bedrock — least privilege
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["bedrock:InvokeModel"],
    "Resource": [
      "arn:aws:bedrock:ap-south-1::foundation-model/anthropic.claude-3-5-sonnet-20241022-
v2:0"
    ]
  }]
}
```

Incident Triage Lambda — Full Implementation

```
# lambda/incident-triage/handler.py
import boto3, json
```

```
def handler(event, context):
    alarm  = event["detail"]["alarmName"]
    region = event["region"]
    logs   = get_recent_logs(region, minutes=10)

    bedrock = boto3.client("bedrock-runtime", region_name=region)

    prompt = f"""
    Alert fired: {alarm}
    Recent logs (last 10 minutes): {logs}

    Answer these three questions:
    1. What likely caused this alert?
    2. What should the on-call engineer check first?
    3. List exactly 3 diagnostic commands to run right now.
    Keep your response under 200 words."""

    response = bedrock.invoke_model(
        modelId="anthropic.claude-3-5-sonnet-20241022-v2:0",
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 400,
            "messages": [{"role": "user", "content": prompt}]
        }),
        contentType="application/json",
        accept="application/json"
    )

    analysis = json.loads(response["body"].read())["content"][0]["text"]
    post_to_slack(alarm, analysis)  # include AI analysis in Slack alert
    return {"status": "ok"}

# Critical rules:
# 1. AI is advisory only — never automate remediation without human approval
# 2. Log every Bedrock call for audit trail
# 3. Never send PII or customer data to Bedrock
```

Security and Compliance

RBAC, secrets, encryption, audit trails

Security Checklist

Control	Implementation	How to Verify
Secrets never in code	K8s secrets or AWS Secrets Manager via IRSA	Gitleaks scan in CI pipeline — zero findings
TLS everywhere	OTel Gateway uses TLS. New Relic endpoint is HTTPS.	curl -v to gateway endpoint — shows TLS handshake
IRSA for AWS auth	Each ServiceAccount has scoped IAM role via OIDC	aws sts get-caller-identity inside pod confirms scoped role
Namespace isolation	Teams see only own namespace in NR dashboards	Log in as team user and verify data scope
Least privilege RBAC	NR user roles match RBAC table in Chapter 4	NR → User management → review assigned roles
Audit logging	All NR API calls logged. CloudTrail for AWS.	NR → Audit log → query recent actions
Data residency	Bedrock in same region as workloads	Check Bedrock region in AWS console
No PII in telemetry	OTel processor strips PII fields before export	Grep spans for email/phone patterns
License key rotation	NR license key rotated every 90 days	Calendar reminder + automation script

Compliance Evidence — NRQL Queries for Auditors

These queries produce audit evidence for SOC 2, ISO 27001, and HIPAA assessments. Run them in New Relic Query Builder and export the results.

Who accessed observability data and when

```
SELECT timestamp, actionName, actorEmail, targetName
FROM NrAuditEvent
SINCE 90 days ago
ORDER BY timestamp DESC
```

Alert policy changes — change management evidence

```
SELECT timestamp, actorEmail, actionName, description
FROM NrAuditEvent
WHERE actionName LIKE '%alert%'
SINCE 30 days ago
```

Data access by user — access control evidence

```
SELECT count(*) as accessCount, actorEmail
FROM NrAuditEvent
SINCE 30 days ago
```

```
FACET actorEmail
ORDER BY accessCount DESC
```

Operations Runbook

Day-2 operations, scaling guidelines, troubleshooting

Daily Operations Checklist

Check	Command or Location	Expected Result
Gateway pods healthy	kubectl get pods -n observability | grep gateway	3/3 Running
Agent pods healthy	kubectl get pods -n observability -l app=otel-agent	1 per node, all Running
NR data flowing	SELECT count(*) FROM Metric SINCE 5 minutes ago	Returns > 0 results
Synthetic monitors	NR → Synthetics → Overview page	All monitors green
Alert queue	NR → Alerts → Active alerts	0 critical, review any warnings
Gateway queue size	average(otelcol_exporter_queue_size) FROM Metric	Below 1000 (below warning)

Common Issues and Fixes

Issue: No data in New Relic after 10 minutes

```
kubectl logs -n observability deploy/otel-gateway --tail=50
# Look for: 'Failed to export' or 'connection refused'

kubectl get secret newrelic-license -n observability -o yaml
# Verify license key is set and correct (decode base64 to check)

kubectl describe configmap otel-gateway-config -n observability
# Verify endpoint is: https://otlp.nr-data.net
```

Issue: OTel Agent in CrashLoopBackOff

```
kubectl describe pod -n observability -l app=otel-agent
# Check Events section — usually OOM kill or config YAML error

kubectl logs -n observability daemonset/otel-agent --previous
# Previous container logs show the exact crash reason
```

Issue: Gateway memory pressure

```
kubectl top pods -n observability
# If Gateway using > 1.5Gi, reduce batch size in ConfigMap:
# batch.send_batch_size: 512  (reduce from 1024)
# Or increase memory limit: limits.memory: '3Gi'
```

Issue: Synthetic monitor failing from one location only

```
# Usually a firewall rule — not an application problem
# Allow New Relic synthetic runner IPs in your load balancer security group
# IP list: docs.newrelic.com/docs/synthetics/.../synthetic-public-minion-ips
```

Scaling Guidelines

Signal	Scale When	Action
Gateway CPU > 70% sustained	CPU above threshold for 10+ minutes	Add replica: kubectl scale deploy/otel-gateway --replicas=4
Gateway queue > 3000	Queue size exceeds warning threshold	Increase sending_queue.queue_size in ConfigMap, rolling restart
Agent OOMKilled	Agent container killed by OOM	Increase agent memory limit to 768Mi in DaemonSet
NR ingest near 100GB	Approaching free tier limit	Increase tail sampling drop rate — reduce normal to 5%
New cluster onboarding	Adding a new cloud cluster	Copy DaemonSet, update cloud_provider tag, apply to new context

Saroj Naik

Principal Cloud Architect

20+ years in enterprise IT, cloud infrastructure, and platform engineering

Areas of Expertise

Cloud Architecture (AWS · Azure · GCP) | DevSecOps | Zero-Trust Security
Multi-Cloud Observability | Kubernetes | GitOps | Platform Engineering
AI-Powered Operations | Amazon Bedrock | DevSecOps Automation

End-to-end cloud platform design and implementation
Free initial evaluation and TCO analysis available

linkedin.com/in/nsaroj

Connect on LinkedIn to discuss your observability platform needs

This guidebook is provided as a professional reference. All configurations are production-tested patterns.